

Deterministic Testing — Simulator & DAG Builder

How the project reproduces whole-network scenarios with no real time, network, or mining — a controlled-clock simulator and a DAG you can write in text.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 43 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Simulator · Deterministic testing · Controlled clock · Fake reactor · In-memory nodes ·
FakeConnection · DAG builder · DSL · Reproducibility · Seeded randomness

Table of Contents

Chapter 43 — Deterministic Testing: Simulator & DAG Builder	3
43.1 Localization	3
43.2 What it does, and why it exists	4
The problem: timing-dependent bugs	4
The cure: determinism	5
43.3 The concepts it rests on	6
43.4 The code, walked — the key trick first	7
Generic warm-up: advance time by hand	7
Hathor's spelling: HeapClock	8
43.5 The code, walked — the Simulator class	9
Building an in-memory node	9
Wiring two nodes together: FakeConnection	10
Running the simulation: stepping time	11
43.6 The code, walked — producing blocks and transactions without mining	12
43.7 The code, walked — writing a DAG in text	13
The DSL	13
Tokenize, then assemble	14
The result: DAGArtifacts	14
43.8 How it plugs into the lifecycle	15
Recap	16

Chapter 43 — Deterministic Testing: Simulator & DAG Builder

What you'll learn in this chapter

- Why consensus and sync bugs are timing-dependent, why that makes them rare and hard to debug against a real network, and how **determinism** turns a flaky bug into a fixed test.
- The single key trick the simulator rests on: **advance time by hand** instead of sleeping — a fake reactor whose clock you step manually.
- How `hathor/simulator/` builds several **in-memory nodes**, wires two of them together with `FakeConnection`, and drives them all from one stepped clock with `run(...)`.
- How blocks and transactions are produced without real mining — a geometric model that fires a `callLater` instead of grinding a nonce.
- How `hathor/dag_builder/` lets a test **write a DAG in a compact text language** (a DSL) and get back real, signed vertices.
- Where all of this plugs into the test suite, and how it exercises consensus, sync, and the full ingestion pipeline — deterministically.

This is the last chapter of Part II, and it is a fitting place to end. Every earlier chapter walked one moving part of the node: the vertex model, storage, consensus, the peer-to-peer layer, sync. This chapter is about the machinery the project uses to test all of those parts together — to spin up a small network of nodes inside a single test process, make them mine, talk, conflict, and sync, and check that the right thing happened. And it does so without ever touching a real clock, a real socket, or a real proof-of-work puzzle.

Two packages cooperate. `hathor/simulator/` runs time: it stands up in-memory nodes, connects them, and advances a controlled clock so that hours of network activity replay in milliseconds. `hathor/dag_builder/` runs structure: it lets a test describe a precise DAG — "blocks b1..b50, transaction tx50 spends tx10 and tx20, block b33 confirms tx50" — as a few lines of text, and turns that description into real vertices the node can ingest.

43.1 Localization

Both packages live under `hathor/`, in the infrastructure tier of the module map (Chapter 0, §0.4). Neither ships in the running node's hot path; they exist to serve tests and tools.

hathor-core/	
└─ hathor/	
└─ simulator/	◀ YOU ARE HERE (time)
__init__.py	← re-exports Simulator, FakeConnection, ...
simulator.py	← the Simulator class: peers, clock, run()
clock.py	← HeapClock + MemoryReactorHeapClock (fake reactor)
fake_connection.py	← FakeConnection: wire two managers in memory
tx_generator.py	← RandomTransactionGenerator (no mining)
trigger.py	← stop conditions for a run
patches.py	← skip real PoW in the simulator
utils.py	← helpers to build txs / double-spends
miner/	
abstract_miner.py	← AbstractMiner base
geometric_miner.py	← GeometricMiner: blocks via geometric timing
└─ dag_builder/	◀ YOU ARE HERE (structure)
builder.py	← DAGBuilder: parse + assemble the DAG
tokenizer.py	← the DSL grammar + tokenizer (DSL docstring here)
types.py	← DAGNode, DAGInput, DAGOutput
artifacts.py	← DAGArtifacts: the built vertices + propagate_with
default_filler.py	← fills in unspecified amounts/parents
vertex_exporter.py	← turns the abstract DAG into real vertices
cli.py	← build a DAG from a file, print hex
utils.py / types.py	← small parsing helpers / type aliases

Context. A full node is a concurrent, networked program. Most of the behaviour worth testing — does a reorg resolve correctly, do two conflicting transactions void the right one, do two peers converge on the same ledger — only appears when several nodes run over time and exchange data. You cannot reliably reproduce that against the live network: it is slow, non-repeatable, and you don't control it. The simulator package recreates the whole network inside one Python process and one controllable clock; the dag_builder package lets a test state the exact graph it wants to reason about. Together they are how the project makes the rest of the book's machinery testable.

43.2 What it does, and why it exists

The problem: timing-dependent bugs

Recall what the node actually does in steady state (Chapter 0, §0.3). It sits in an event loop and reacts: a peer connects, a block arrives, a timer fires, a transaction is broadcast. The order in which those events land is not fixed. Two peers, each mining, each relaying — the exact interleaving of "block from peer A arrives" versus "transaction from peer B arrives" versus "my own mining timer fires" depends on network latency, CPU speed, and luck.

Most node code is correct regardless of that order. But the hardest, most consequential code is not. Consensus (Chapter 32) decides which of two conflicting transactions wins and which is voided; sync (Chapter 35) brings a lagging peer up to date over many round-trips; a reorg¹ rewrites which chain of blocks is canonical when a heavier one appears. Bugs in these subsystems are almost always **timing-dependent**: they only manifest under one particular interleaving of events. Run the same scenario again and the interleaving shifts, and the bug hides.

A timing-dependent bug that you cannot reproduce on demand is nearly impossible to fix. You see it once in a log, you change something, you can't tell whether you fixed it or merely got lucky. This is the single worst category of bug in a networked system, and consensus is exactly where it lives.

The cure: determinism

The fix is **determinism**: arrange things so that the same test always produces the same sequence of events, every single run. If a bug appears once, it appears every time; once you can reproduce it on demand, you can bisect it, fix it, and lock the fix in as a regression test. A flaky, once-in-a-thousand-runs failure becomes a permanent, always-failing test that you then make pass — and it stays passing forever after.

So where does non-determinism come from, and how does the simulator remove each source?

Source of non-determinism	In the real node	In the simulator
Wall-clock time	timers fire when real seconds pass	a fake clock you advance by hand (§43.4)
Network latency / OS scheduling	real sockets, unpredictable delays	<code>FakeConnection</code> ferries bytes in-memory, step by step (§43.5)
Proof-of-work	grinding a nonce takes random real time	PoW is skipped; block timing comes from a seeded model (§43.6)
Randomness	system entropy (addresses, choices)	one seeded random generator (§43.3)
Disk / OS state	persistent RocksDB on disk	a throwaway temporary database (§43.3)

Remove all five and the simulation is **reproducible**: a given seed plus a given scenario always replays identically. That is the whole game.

A worked analogy before any code. Imagine a board game you want to test for a rules bug. Played with real friends, real dice, and a real timer, every play-through differs — you can never re-create the exact game where the bug showed up. Now imagine a version where you roll the dice from a pre-recorded list (seeded randomness), nobody waits on a clock — you just call out "next turn" yourself (the controlled clock), and the board is a scratch copy you throw away after (temporary storage). Now every play-through with the same dice-list is identical. If a rules bug appears, it appears every time. That is precisely what the simulator does for a network of nodes.

43.3 The concepts it rests on

Four ideas from earlier chapters carry this one. None is re-taught here; each gets a short recap and a pointer.

Recap — the Twisted reactor (full treatment in Ch. 16; abstraction in Ch. 23).

The node is built on Twisted, whose centrepiece is the **reactor**: a single event loop that waits for things to happen (network data, expired timers) and calls your code in response. Code that wants to run "later" does not call `time.sleep`; it asks the reactor `callLater(delay, fn)` and returns. The reactor decides when `fn` actually runs. The simulator's entire trick is to swap the real reactor for a **fake** one whose notion of "now" is a variable you control — so "later" arrives exactly when you say it does.

Recap — sync between peers (full treatment in Ch. 35). When a node is behind, a sync agent on each connection downloads the missing blocks and transactions in dependency order until the two peers hold the same ledger. The protocol is sync-v2; it runs over many message round-trips. Testing it means running two nodes against each other until both report "synced" — the simulator exists in large part to make that loop deterministic.

Recap — temporary, throwaway storage (full treatment in Ch. 27). Production nodes persist the ledger to RocksDB on disk. For tests, the builder can hand a node a temporary database that is created fresh and discarded afterward, so no test sees state left by another. There is no pure in-RAM backend; "in-memory node" here means a node whose storage is a throwaway temp database and whose reactor is fake — the node object itself is a real `HathorManager`.

Recap — the test suite (full treatment in Ch. 20). The project's tests run under `pytest`. The simulator and `dag_builder` are test infrastructure: they are imported by test cases (and a couple of CLI tools), not by the running node. A simulator-based test typically inherits a base class that constructs a `Simulator`, prints its seed, and tears it down afterward.

One more building block, local to this chapter: **seeded randomness**. The simulator creates exactly one random-number generator from a seed and threads it through everything that needs a random choice — wallet words, transaction values, mining timing. The generator is `hathor.util.Random`, a subclass of Python's `random.Random` that adds a `geometric(p)` method (`util.py:264`). Because every node and generator descends from that one seed (`simulator.py:55-58`), re-running with the same seed reproduces every "random" decision exactly. When a seed is not given, one is drawn from `secrets` and printed, so a failing run can be replayed by pasting the seed back in (the base test case prints it: `hathor_tests/simulation/base.py:18-20`).

43.4 The code, walked — the key trick first

Generic warm-up: advance time by hand

Forget Hathor for a moment. Suppose you are testing a plain function that schedules work for later:

```
def remind_me(scheduler, seconds, message):
    scheduler.call_later(seconds, lambda: print(message))
```

If `scheduler` is the real system clock, a test must actually wait `seconds` to see the message — slow, and never exactly repeatable. The standard fix is a **fake scheduler** that does not wait at all. It keeps a list of `(when, callback)` pairs and a variable `now`. Calling `call_later` just appends to the list. Nothing runs until the test says so:

```
class FakeScheduler:
    def __init__(self):
        self.now = 0.0
        self.calls = []  # (fire_time, callback)

    def call_later(self, delay, fn):
        self.calls.append((self.now + delay, fn))

    def advance(self, amount):  # the test drives time itself
        self.now += amount
        for when, fn in sorted(self.calls):
            if when <= self.now:
                fn()  # "later" has arrived – run it now
```

The test is now instant and deterministic:

```
sched = FakeScheduler()
remind_me(sched, 10, "hello")
# nothing printed yet
sched.advance(10)  # -> prints "hello", with zero real waiting
```

That is the whole idea. **Instead of sleeping, the test advances a clock by hand**, and whatever was scheduled to run "in 10 seconds" runs the instant the clock crosses that mark. No wall-clock time passes; the order is fixed.

Hathor's spelling: HeapClock

Hathor's fake clock is exactly this idea, made to satisfy Twisted's reactor interface so it can stand in for the real reactor. It lives in `hathor/simulator/clock.py`. The class `HeapClock` (`clock.py:25`) declares itself a Twisted `IReactorTime` implementation and holds:

- `rightNow` — the current simulated time, a single float (`clock.py:32`). This is the fake "now."
- `calls` — a heap (priority queue) of pending `(fire_time, DelayedCall)` pairs (`clock.py:35`).

Its `seconds()` returns `rightNow` — the node, asking "what time is it?", gets the simulated answer, not the system answer (`clock.py:37-45`). Its `callLater` is our `call_later`: build a `DelayedCall`, push it on the heap keyed by fire-time, return (`clock.py:47-57`). And the engine is `advance`:

```
def advance(self, amount):
    assert amount >= 0
    self.rightNow += amount
    while self.calls:
        time, call = self.calls[0]      # smallest fire-time on the heap
        if time > self.seconds():
            break                        # nothing else is due yet
        heapq.heappop(self.calls)
        if not call.cancelled:
            call.called = 1
            call.func(*call.args, **call.kw)
```

`hathor/simulator/clock.py:65-82` — moving time forward runs everything now due, in time order.

A heap² is used so that "the next thing to fire" is always cheap to find: the earliest fire-time sits at the front. `advance` pops and runs every call whose fire-time is at or before the new `rightNow`, in chronological order, then stops. Time only moves when the simulation chooses to move it.

The class actually used is `MemoryReactorHeapClock` (`clock.py:93`), which mixes `HeapClock` together with Twisted's `MemoryReactor` — a test reactor that fakes network connections in memory rather than opening real sockets. Note the deliberate override of `run()` (`clock.py:98-104`): the stock `MemoryReactor.run()` immediately calls `stop()`, which would be wrong here, so Hathor's version just sets `running = True` and leaves the reactor "running" for the duration of the test.

This object is the determinism. It is a real, interface-compatible reactor as far as the node knows — but its clock is a variable the test steps by hand.

43.5 The code, walked — the Simulator class

`Simulator` (`hathor/simulator/simulator.py:52`) is the conductor. Constructing one (`simulator.py:53-65`) does four things that matter:

1. Picks a **seed** — given or drawn from `secrets` — and builds the one seeded `Random` from it (`simulator.py:55-58`).
2. Copies the global settings but **shortens the average time between blocks** to 64 seconds (`simulator.py:59-61`, constant `SIMULATOR_AVG_TIME_BETWEEN_BLOCKS = 64` at `:49`), so simulated chains grow fast.
3. Creates the fake reactor: `self._clock = MemoryReactorHeapClock()` (`simulator.py:62`).
4. Prepares empty registries for peers and connections (`simulator.py:63-64`).

`start()` (`simulator.py:67-74`) must be called before anything else. It nudges the clock forward by a randomized amount past the genesis timestamp — `dt = self.rng.randint(3600, 120 * 24 * 3600)` (`simulator.py:72-73`) — so that tests don't all begin at the exact same instant. Because `dt` comes from the seeded generator, it is still reproducible.

Building an in-memory node

`create_peer(builder)` (`simulator.py:91-97`) returns a fully-wired `HathorManager` — a real node object (Chapter 29) — configured for simulation. The heavy lifting is in `create_artifacts` (`simulator.py:99-133`). It takes a `Builder` (Chapter 24, the composition root) and overrides the pieces that would otherwise reach for real time, real PoW, or real entropy:

```
artifacts = builder \
    .set_reactor(self._clock) \                # the fake reactor, not the real one
    .set_rng(Random(self.rng.getrandbits(64))) \    # a seeded child RNG
    .set_wallet(wallet) \
    .set_vertex_verifiers_builder(_build_vertex_verifiers) \    # PoW-skipping verifier
    .set_daa(daa) \
    .set_cpu_mining_service(cpu_mining_service) \    # PoW-skipping miner
    .build()

artifacts.manager.start()
self._clock.run()
self.run_to_completion()
```

`hathor/simulator/simulator.py:113-124` — every non-deterministic dependency is replaced before `build()`.

Three substitutions deserve a name. `set_reactor(self._clock)` is the core swap — the node's reactor is the fake clock, so every `callLater` the node makes lands on our heap. `set_rng(...)` gives the node a child generator derived from the master seed, so even

the node's internal random choices are reproducible. And the two PoW-skippers come from `hathor/simulator/patches.py: SimulatorVertexVerifier.verify_pow ( patches.py: 27-30 )` does nothing — it logs "skipping" and returns — and `SimulatorCpuMiningService.resolve ( patches.py:33-43 )` just recomputes the hash without grinding for a valid nonce. The simulation does not need real proof-of-work; it needs timing that resembles mining, which it gets elsewhere (§43.6).

After `start()`, the call `self._clock.run()` flips the reactor to "running" and `run_to_completion()` (`simulator.py:141-148`) drains any setup work the node scheduled during boot by advancing the clock to each pending call's time. The node is now alive and idle, waiting for the test to move time.

Built nodes are registered by name with `add_peer(name, manager)` (`simulator.py: 149-153`) and fetched with `get_peer(name)` (`simulator.py:158-159`).

Wiring two nodes together: FakeConnection

A real connection between two nodes is a TCP socket: bytes one node writes appear, eventually, at the other. `FakeConnection` (`hathor/simulator/fake_connection.py:50`) replaces the socket with an in-memory shuttle you step by hand.

Its constructor (`fake_connection.py:54-89`) takes the two managers, an optional `latency` in seconds, and immediately calls `reconnect()` (`fake_connection.py:268-292`), which builds a protocol object on each side from the managers' real connection factories and joins them through `HathorStringTransport` — a Twisted test transport that captures written bytes into a buffer instead of sending them to an OS socket. Crucially, this exercises the real peer-to-peer protocol code (handshake, sync agent, message dispatch from Chapters 34-35); only the wire underneath is fake.

The pump is `run_one_step` (`fake_connection.py:195-245`). Each call takes whatever bytes node 1 has buffered to send, and delivers them to node 2 — and vice versa:

```
if line1:
    if self.latency > 0:
        self.manager1.reactor.callLater(          # deliver after a fake delay
            self.latency, self._deliver_message, self._proto2, line1, debug)
    else:
        self._proto2.dataReceived(line1)         # hand node 1's bytes straight to node 2
```

`hathor/simulator/fake_connection.py:222-230` — one message hop, optionally delayed by a fake latency.

When `latency > 0`, delivery is scheduled on the fake clock with `callLater(latency, ...)` instead of happening at once — so a test can model a slow link, and the message arrives only after the clock advances past that latency. This is how the simulator reproduces network-delay-dependent ordering deterministically: the "delay" is fake-clock seconds, not real ones.

Two helper predicates make step-loops readable without guessing iteration counts:

`is_both_synced()` (`fake_connection.py:119-164`) returns `True` only when both protocols are in the `ReadyState`, neither errored, both sync agents report synced, and both peers agree on the best block and the mempool tips. `can_step()` (`fake_connection.py:166-193`) reports whether there is still useful work to do on the connection — the run loop uses it to know when to stop.

A connection is registered with the simulator via `add_connection(conn)` (`simulator.py:161-162`) so the run loop will pump it.

Running the simulation: stepping time

`run(interval, step=0.25, ...)` (`simulator.py:225-242`) is the main entry point: advance simulated time by `interval` seconds, in increments of `step`, pumping every connection at each increment. The work is in the private generator `_run` (`simulator.py:167-198`):

```
while self._clock.seconds() <= initial + interval:
    for conn in self._connections:
        conn.run_one_step()      # ferry one message hop per connection
    yield
    ...
    self._clock.advance(step)    # move the fake clock forward one step
```

`hathor/simulator/simulator.py:174-198` — the heartbeat: step the connections, advance time, repeat.

Each loop iteration: deliver one round of messages across every connection, then advance the clock by `step` (default 0.25 s, `DEFAULT_STEP_INTERVAL` at `simulator.py:47`), which fires any node timers now due — mining timers, sync's periodic `LoopingCall`, transaction generation. Repeat until `interval` simulated seconds have elapsed. Real wall-clock time barely moves; simulated time marches in fixed steps. The same `interval` and `step` with the same seed produce byte-for-byte the same run.

Two siblings refine this. `run_until_complete(max_interval, ...)` (`simulator.py:200-223`) steps until every connection reports it `can_step()` no more — i.e. all peers have synced (or errored) — returning `True`, or until `max_interval` is exhausted, returning `False`. That lets a sync test say "run until they converge" without hard-coding how many steps convergence takes. And `run(..., trigger=...)` accepts a **Trigger** (`hathor/simulator/trigger.py:28`), an object with a `should_stop()` method, checked each iteration; the run ends when the condition holds. Concrete triggers include `StopAfterNMinedBlocks` (`trigger.py:36`), `StopAfterNTransactions` (`trigger.py:69`), `StopWhenSynced` (`trigger.py:94`), and `All` to combine them (`trigger.py:103`).

43.6 The code, walked — producing blocks and transactions without mining

A network is dull without traffic. The simulator can generate both blocks and transactions, and it does so without real proof-of-work — instead it models how long mining would take and schedules the result on the fake clock.

The statistical idea is the **geometric distribution**³. Mining is repeated independent trials, each with a tiny success probability `p` determined by the block's weight: `p = 2**(-weight)`. The number of trials until the first success follows a geometric distribution with parameter `p`. So instead of actually trying nonces, the simulator draws a trial count from that distribution and divides by a configured `hashpower` to get a time delay — then schedules the block to "appear" after that many fake seconds. Statistically, blocks arrive with the same timing they would under real mining, but no CPU is spent and the outcome is reproducible from the seed.

`GeometricMiner` (`hathor/simulator/miner/geometric_miner.py:30`) implements this. Its `_schedule_next_block` (`geometric_miner.py:84-121`) is the loop. When a block is "found," it sets a random nonce, updates the hash, and propagates it through the node's real ingestion path:

```
if self._block:
    self._block.nonce = self._rng.getrandbits(32)
    self._block.update_hash()
    self._manager.propagate_tx(self._block)      # enters the real pipeline (Ch 33)
    self._blocks_found += 1
    self._block = None
...
geometric_p = 2**(-block.weight)
trials = self._rng.geometric(geometric_p)        # how many "tries" mining would take
dt = 1.0 * trials / self._hashpower              # convert tries -> fake seconds
self._block = block
self._delayed_call = self._clock.callLater(dt, self._schedule_next_block)
```

`hathor/simulator/miner/geometric_miner.py:89-121` — block timing drawn from a seeded geometric model, scheduled on the fake clock.

The miner subscribes to `NETWORK_NEW_TX_ACCEPTED` so that when a competing block arrives it can abandon its current candidate and start over on the new chain head (`abstract_miner.py:43-47`, `geometric_miner.py:56-70`) — exactly the race a reorg test wants to provoke. A test creates one with `simulator.create_miner(peer, hashpower=...)` (`simulator.py:138-139`).

`RandomTransactionGenerator` (`hathor/simulator/tx_generator.py:36`) does the analogous job for transactions: on a seeded schedule it picks a wallet address and amount, builds a real transaction, sets its minimum weight, draws a geometric "mining" delay, and propagates it (`tx_generator.py:93-148`). The inter-transaction gap is `self.rng.expovariate(self.rate)` (`tx_generator.py:105`) — an exponential wait, the natural model for "events arriving at an average rate" — again drawn from the seeded

generator. It can even be told to emit deliberate double-spends (`tx_generator.py:90-91, 131-137`) to test conflict resolution. A test creates one with `simulator.create_tx_generator(peer, rate=..., hashpower=...) ( simulator.py:135-136 )`.

So the traffic on the simulated network — when blocks land, when transactions fire, who conflicts with whom — is entirely a function of the seed. Replay the seed, replay the traffic.

43.7 The code, walked — writing a DAG in text

The simulator runs time. The `dag_builder` builds structure. Sometimes a test does not want a random network at all; it wants a specific graph — "block b33 confirms transaction tx50, which spends outputs of tx10 and tx20" — so it can assert exactly what consensus does with it. Constructing such a graph by hand in Python is verbose and error-prone: you must create each output, wire each input to the right output index, set parents, sign everything. The `dag_builder` lets you write that graph as a few lines of a small **domain-specific language**⁴ (DSL) and get back real, signed vertices.

The DSL

The grammar is documented in the module docstring at the top of `hathor/dag_builder/tokenizer.py:20-129`. The core statements:

```
blockchain genesis b[1..50]    # create blocks b1..b50, chained, b1's parent is genesis
a <-- b <-- c                  # a is a parent of b, which is a parent of c
a --> b --> c                  # the same edges, written the other direction
a.out[0] <<< b c                # b and c each spend output 0 of a
a < b < c                      # ordering: a must be created before b before c
a.out[0] = 100 HTR [wallet1]    # output 0 of a holds 100 HTR, owned by wallet1
a.weight = 50                   # set a vertex attribute
dummy                           # auto-created tx spending genesis, to fund user txs
```

Here is a real example lifted from a consensus test (`hathor_tests/consensus/test_first_block.py:23-43`):

```
blockchain genesis b[1..50]

b30 < dummy

tx10.out[0] <<< tx50
tx20.out[0] <<< tx50
tx30 <-- tx50
tx40 <-- tx50

b31 --> tx10
b32 --> tx30
b33 --> tx50
```

Read top to bottom: make a 50-block chain off genesis; place a funding `dummy` transaction after block 30 (so its outputs clear the reward lock); transaction `tx50` spends output 0 of `tx10` and `tx20` and lists `tx30` and `tx40` as parents; and blocks `b31`, `b32`, `b33` confirm `tx10`, `tx30`, `tx50` respectively. A test can then fetch `tx50` and assert which block first confirmed it — a precise, readable consensus scenario in ten lines.

Tokenize, then assemble

`build_from_str(content)` (`hathor/dag_builder/builder.py:353-356`) is the entry point. It runs in two passes. First, `tokenize` (`tokenizer.py:164-262`) reads the text line by line, strips comments at `#`, and emits a stream of typed tokens — `BLOCKCHAIN`, `PARENT`, `SPEND`, `OUTPUT`, `ORDER_BEFORE`, `ATTRIBUTE` (the `TokenType` enum, `tokenizer.py:134-141`). For example a `<--` line is split into pairs and each becomes a `PARENT` token (`tokenizer.py:232-234`); an `a.out[i] <<< ...` line becomes one `SPEND` token per spender (`tokenizer.py:240-248`).

Second, `parse_tokens` (`builder.py:105-128`) consumes that stream with a `match` statement and mutates an in-memory graph of `DAGNode` objects (`types.py:42-57`). Each node records its parents, inputs (`DAGInput` = source node + output index, `types.py:99-101`), outputs (`DAGOutput` = amount + token + attributes, `types.py:104-107`), and dependencies. A `SPEND` token, for instance, ensures the spent output exists and adds a `DAGInput` to the spender (`builder.py:187-195`).

`build()` (`builder.py:348-351`) then finishes the job in two final steps. A `DefaultFiller` fills in anything the test left unspecified — output amounts that balance, missing parents — so the test only states what it cares about. Then a `VertexExporter` walks the graph in **topological order** (`topological_sorting`, `builder.py:310-346` — a dependency-respecting order, the same idea as a DAG toposort from Chapter 8) and turns each abstract `DAGNode` into a real `Block` or `Transaction`, allocating wallets, building outputs, wiring inputs to the right output bytes, and signing. If the description contains a cycle, the toposort detects it and raises (`builder.py:326-336`).

The result: DAGArtifacts

The output is a `DAGArtifacts` (`hathor/dag_builder/artifacts.py:33`). It holds the built vertices both as an ordered tuple and a name-keyed map (`artifacts.py:34-43`), so a test can write `artifacts.by_name['tx50']` to get the real vertex it described. Two conveniences matter:

- `get_typed_vertex(name, Block)` (`artifacts.py:51-55`) fetches a vertex and asserts its type, so the test gets a correctly-typed object.
- `propagate_with(manager, up_to=...)` (`artifacts.py:61-103`) feeds the vertices, in built order, into a real node through `vertex_handler.on_new_relayed_vertex` (`artifacts.py:88`) — the same ingestion pipeline from Chapter 33 — optionally stopping at a named vertex. This is how a test moves from "a

described DAG" to "a node that has actually ingested that DAG," ready to be queried.

Tests usually construct the builder via a small wrapper,

`TestDAGBuilder.from_manager(manager)` (`hathor_tests/dag_builder/builder.py:28`), which calls the real `DAGBuilder.from_manager` (`builder.py:83-100`) with a test wallet and blueprint module. There is also a standalone CLI, `hathor/dag_builder/cli.py`, that reads a DSL file and prints each resulting vertex as hex (`cli.py:18-61`) — handy for generating fixtures.

43.8 How it plugs into the lifecycle

Neither package runs inside a production node. Both are imported by the test suite (Chapter 20) and by two small CLI tools.

A simulator-based test typically inherits `SimulatorTestCase` (`hathor_tests/simulation/base.py:8`), which builds a `Simulator`, prints its seed, and tears it down. From there a sync test reads almost like prose: create two peers, connect them with a `FakeConnection`, register it, and `run_until_complete` — then assert both are synced. The real sync test does exactly this (`hathor_tests/simulation/test_simulator.py:48-49` create and register a `FakeConnection` with 150 ms fake latency; later loops add auto-reconnecting connections to model flapping links). Because every input is seeded and time is stepped, a failure prints a seed that reproduces it forever.

Through that one mechanism the simulator exercises the heaviest machinery in the book, deterministically:

- **Consensus (Chapter 32)** — generators emitting conflicting transactions and competing blocks let tests assert exactly which vertex is voided and which chain wins after a reorg. The soft-voided-transaction tests (`hathor_tests/consensus/test_soft_voided*.py`) are built on the simulator.
- **Sync (Chapter 35)** — two or more peers run their real sync-v2 agents over `FakeConnection`s until `is_both_synced()` holds; split-brain and late-joiner scenarios are modelled with latency and reconnection (`hathor_tests/p2p/test_sync.py`, `test_split_brain.py`).
- **The full ingestion pipeline (Chapter 33)** — every block and transaction the simulator or `dag_builder` produces enters the node through the same `vertex_handler` path the live node uses (verify → consensus → store), so the test exercises the real code, not a mock.
- **Feature activation (Chapter 38)** — the geometric miner accepts a list of `signal_bits` per block (`geometric_miner.py:40-51`, `72-82`), so a test can drive the bit-signalling schedule deterministically (`hathor_tests/feature_activation/test_feature_simulation.py`).

The two CLI tools are thin: `hathor_cli/events_simulator/` runs a named scenario on a `Simulator` and forwards the resulting events over a WebSocket — used to develop event-queue clients against a reproducible event stream. And `hathor/dag_builder/cli.py` turns a DSL file into hex vertices for fixtures.

This is where the whole book's machinery gets tested. Every subsystem walked in Part II — the vertex model, storage, verification, consensus, the peer-to-peer layer, sync — meets here, run together, against a clock you control, with bugs that stay reproducible.

Recap

Piece	File	Role
<code>HeapClock / MemoryReactorHeapClock</code>	<code>simulator/clock.py:25, 93</code>	fake reactor; <code>advance()</code> steps simulated time by hand
<code>Simulator</code>	<code>simulator/simulator.py:52</code>	builds peers, holds the clock, <code>run()</code> steps the world
<code>FakeConnection</code>	<code>simulator/fake_connection.py:50</code>	wires two managers in memory; <code>run_one_step()</code> ferries bytes
PoW patches	<code>simulator/patches.py: 27, 33</code>	skip real proof-of-work; hashes recomputed, not ground
<code>GeometricMiner</code>	<code>simulator/miner/geometric_miner.py:30</code>	blocks via a seeded geometric timing model
<code>RandomTransactionGenerator</code>	<code>simulator/tx_generator.py:36</code>	transactions on a seeded schedule, optional double-spends
<code>Trigger</code> family	<code>simulator/trigger.py: 28</code>	stop conditions: N blocks, N txs, synced, ...
The DSL	<code>dag_builder/tokenizer.py:20</code>	text grammar for describing a DAG
<code>DAGBuilder</code>	<code>dag_builder/builder.py:52</code>	tokenize → assemble nodes → export real vertices

Piece	File	Role
DAGArtifacts	<code>dag_builder/artifacts.py:33</code>	the built vertices; <code>propagate_with()</code> ingests them

Determinism is the whole point. Wall-clock time becomes a variable you step; the network becomes an in-memory shuttle you pump; proof-of-work becomes a seeded timing model; randomness and storage collapse to one seed and one throwaway database. With all five sources of non-determinism removed, a network scenario replays identically every run — and the rarest, nastiest class of bug, the timing-dependent consensus or sync failure, stops being a ghost you glimpse once in a log and becomes a test that fails every time until you fix it. The `dag_builder` complements this by letting a test state the exact graph it wants in a handful of readable lines instead of pages of object construction.

With that, **Part II is complete**. We have walked the node end to end: from the command line that boots it (Chapter 21) and the settings that shape it (Chapter 22), through the vertex model, serialization, storage, and indexes that hold its data (Chapters 25–28); the manager, events, verification, consensus, and ingestion that give it life and rules (Chapters 29–33); the peer-to-peer and sync layers that connect it to the world and the service surfaces it exposes (Chapters 34–37); the higher subsystems of feature activation, nano-contracts, and wallets (Chapters 38–40); the runtime control and observability that keep it operable (Chapters 41–42); and finally, here, the deterministic testing infrastructure that keeps all of it honest. What remains are the **appendices** — the glossary and jargon index (Appendix A), the annotated dependency manifest (Appendix B), and the cheat sheets (Appendix C) — reference material to keep beside you, not new ground to learn.

-
1. Reorg (reorganization) is when the node discovers a heavier chain of blocks than the one it currently treats as canonical, and switches to it — re-deciding which recent blocks and transactions count. Reorgs are the classic timing-dependent scenario, and a prime target for simulator tests. Full treatment in Chapter 32. ↩
 2. A heap is a tree-shaped data structure kept partly ordered so that the smallest (or largest) item is always at the front and cheap to remove. Python's `heapq` gives this over a plain list. The simulator uses a min-heap keyed by fire-time so "the next timer to fire" is always at hand. ↩
 3. The geometric distribution models the number of independent yes/no trials until the first success, when each trial succeeds with probability `p`. Mining is exactly that — repeated nonce attempts, each a long shot — so drawing one geometric sample reproduces "how long mining would have taken" without running the trials. ↩
 4. A domain-specific language (DSL) is a small, purpose-built notation for one narrow job — here, describing a DAG of blocks and transactions — as opposed to a general-purpose language like Python. The `dag_builder` DSL is parsed by a hand-written tokenizer; it is not Python, though a few attribute values are parsed as Python expressions. ↩